

Generative AI for Customer Insights

Enhancing NLP Data Understanding Through AI

Fresh Fare Meal Kits — Customer Service Log Analysis

Final Project Report & Code Explanation

April 2024

1. Executive Summary

This report presents the results of a comprehensive analysis of 20 customer service conversation logs from Fresh Fare Meal Kits, conducted using state-of-the-art Generative AI and Natural Language Processing (NLP) techniques. The analysis leverages Google's FLAN-T5 model for text summarization and reason extraction, alongside a multilingual zero-shot classification model to automatically detect subscription cancellation requests.

Key findings from the analysis:

- 5 out of 20 conversations (25%) involved subscription cancellation requests
- Primary cancellation drivers: delivery delays, poor ingredient quality, missing recipe cards
- Most frequent issue categories: missing ingredients, damaged items, late deliveries
- Word cloud analysis confirms customer concerns cluster around delivery and quality themes

These insights provide the analytics team with an automated, scalable pipeline for ongoing customer service monitoring, enabling data-driven improvements to customer interaction protocols.

2. Dataset Overview

The dataset comprises 20 customer service call logs from Fresh Fare Meal Kits, covering April 3–29, 2024. The raw data was stored in a single unstructured column requiring significant preprocessing before analysis.

Property	Detail
File	Call_Logs.csv
Total Records	20 conversation logs
Raw Column	Single 'Logs' column (unstructured)
Date Range	April 3 – April 29, 2024
Company	Fresh Fare Meal Kits

2.1 Issue Categories Identified

Manual review of the 20 call logs revealed the following issue types:

#	Issue Type	Example
1	Missing Ingredient	Missing fresh basil, missing recipe card
2	Damaged/Expired Item	Expired chicken, damaged package
3	Late/Missing Delivery	2 days past scheduled delivery date
4	Subscription Cancellation	Multiple unhappy customers cancelling
5	Meal Plan Change	Upgrading or modifying subscription plan
6	Allergy/Safety Concern	Nuts in kit despite allergy noted
7	Wrong Language Material	Recipe cards printed in Chinese
8	Customization Request	Larger group dinner party kit
9	Spoiled Food	Salmon fillets completely thawed
10	Plan Update Not Applied	Old meal plan delivered after change

3. Methodology & Task Pipeline

The project was structured into 5 sequential tasks, forming a complete NLP analysis pipeline:

Task	Description	Model/Tool
Task 1	Clean & Prepare Data	pandas, Python regex
Task 2	Conversation Summarization	google/flan-t5-base
Task 3	Cancellation Detection	MoritzLaurer/multilingual-MiniLMv2-L6-mnli-xnli
Task 4	Extract Cancellation Reasons	google/flan-t5-base
Task 5	Word Cloud Visualization	wordcloud, matplotlib

4. Analysis Results

4.1 Conversation Summaries (Task 2)

The FLAN-T5 model successfully generated concise summaries for all 20 conversations. Sample summaries include:

Log #	Date	Summary
0	17/04/2024	Client received a meal kit yesterday, but one of the ingredients was missing.
1	17/04/2024	Client is missing the recipe card for the shrimp linguine with garlic butter sauce.
2	18/04/2024	Client is canceling his meal kit subscription.
3	18/04/2024	Client received his meal kit today, but there appears to be a damaged item.
4	07/04/2024	Client received a meal kit yesterday, but the chicken in it was expired.
5	25/04/2024	Client is unhappy with the food kits.
8	12/04/2024	Client is calling Fresh Fare Meal Kits customer service to cancel her subscription.
9	29/04/2024	Client needs to cancel his subscription to Fresh Fare Meal Kits.
17	29/04/2024	Client is frustrated with the service of Fresh Fare Meal Kits.

4.2 Cancellation Detection (Task 3)

The zero-shot classification model identified 5 conversations as cancellation-related out of 20 total (25% cancellation rate). The model classified each conversation into one of two categories: 'cancellation' or 'other'.

Log #	Date	Cancellation Flag	Summary
2	18/04/2024	TRUE	Client is canceling his meal kit subscription.
5	25/04/2024	TRUE	Client is unhappy with the food kits.
8	12/04/2024	TRUE	Client calling to cancel her subscription.
9	29/04/2024	TRUE	Client needs to cancel subscription.
17	29/04/2024	TRUE	Client frustrated with service.

4.3 Cancellation Reasons (Task 4)

For the 5 flagged cancellation conversations, FLAN-T5 extracted specific reasons why customers chose to cancel:

Log #	Extracted Cancellation Reason
2	The client's last three deliveries have been late/missing, poor ingredient quality.
5	The ingredients are rotten, and the recipes are poorly written.
8	Late deliveries, missing items, and awful quality.
9	The client has some unexpected personal issues (non-service-related cancellation).
17	The constant delays, poor quality ingredients, and unsatisfactory service.

5. Key Business Insights

Based on the automated NLP analysis, the following actionable insights are recommended for the strategy meeting:

5.1 Customer Churn Drivers

- Delivery reliability is the #1 cancellation driver — late or missing deliveries repeatedly mentioned
- Ingredient quality is the #2 driver — expired, damaged, or missing ingredients erode trust
- Recipe card issues (missing, wrong language) create friction even for satisfied customers

5.2 Operational Recommendations

- Implement proactive delivery tracking notifications to reduce 'where is my kit?' calls
- Strengthen quality control checks before shipment — particularly for perishables
- Create a digital recipe card backup system delivered at time of order confirmation
- Train agents to offer retention incentives during cancellation calls

5.3 Agent Performance

All agents demonstrated professional and empathetic responses. No agent-specific performance issues were identified in this dataset. Future analysis with larger datasets should include agent-level sentiment tracking.

6. Line-by-Line Code Explanation

This section provides a detailed explanation of every code cell in the Jupyter Notebook (gen_ai_customer_insights.ipynb).

Task 1: Data Cleaning and Preparation

Cell 1 — Import pandas

```
import pandas as pd
```

Imports the pandas library and assigns it the alias 'pd'. Pandas is the foundational Python library for data manipulation. The alias makes it shorter to reference throughout the code.

Cell 2 — Load the Dataset

```
df = pd.read_csv('Call_Logs.csv')
df.head()
```

Line 1: Reads the CSV file 'Call_Logs.csv' into a pandas DataFrame named 'df'. A DataFrame is a tabular data structure with rows and columns. The file contains 20 call logs in a single unstructured 'Logs' column.

Line 2: Displays the first 5 rows of the DataFrame to give a quick preview of the raw data structure.

Cell 3 — Inspect a Single Log Entry

```
df.Logs[0]
```

Accesses the first element (index 0) of the 'Logs' column. This displays the raw text of the first call log, which includes date, time, and the full agent-client conversation. This is an exploratory step to understand the data format before writing the extraction function.

Cell 4 — Define the extract_info Function

```
def extract_info(df):
    lines = df['Logs'].split('\n')
    date = lines[0].split(': ')[1]
    time = lines[1].split(': ')[1]
    conv = "\n".join([line for line in lines[3:] if line != ""])
    return date, time, conv
```

This function takes a single DataFrame row and parses the unstructured 'Logs' text into three components:

- Line 2: Splits the entire log text at newline characters (\n), creating a list of individual lines.
- Line 3: Extracts the date — takes the first line (e.g., 'Date: 17/04/2024'), splits on ': ', and takes the second part.
- Line 4: Extracts the time — same approach on the second line (e.g., 'Time: 15:45:37').
- Line 5: Extracts the conversation — joins all lines from index 3 onwards (skipping date, time, and empty line), filtering out any blank lines.

- Line 6: Returns all three extracted values as a tuple.

Cell 5 — Apply Function & View Result

```
df.head
```

Displays the `.head` method reference (without parentheses). This was a quick inspection step to confirm the DataFrame structure before applying the extraction function.

Cell 6 — Apply `extract_info` to All Rows

```
df[['Date', 'Time', 'Conversation']] = df.apply(extract_info, axis=1,
result_type="expand")
df.head()
```

Line 1: Applies the `extract_info` function to every row in the DataFrame simultaneously using `.apply()`. The `axis=1` parameter means 'apply function row-by-row' (as opposed to column-by-column). The `result_type='expand'` parameter ensures the returned tuple is automatically unpacked into three separate columns: 'Date', 'Time', and 'Conversation'. These new columns are added directly to the existing DataFrame.

Line 2: Displays the first 5 rows to confirm the new columns were created correctly.

Cell 7 — Remove Unnecessary Columns

```
df.drop(['Logs', 'Unnamed: 0'], axis=1, inplace=True)
df.head()
```

Line 1: Removes two columns that are no longer needed: (1) 'Logs' — the original raw text column, now replaced by the three structured columns; (2) 'Unnamed: 0' — the original index column that was automatically created when reading the CSV. The `axis=1` parameter specifies we are dropping columns (not rows), and `inplace=True` modifies the DataFrame directly without needing to reassign it.

Line 2: Displays the cleaned DataFrame with only 3 columns: Date, Time, and Conversation.

Task 2: Automated Conversation Summarization with FLAN-T5

Cell 8 — Import T5 Classes

```
from transformers import T5Tokenizer, T5ForConditionalGeneration
```

Imports two essential classes from the Hugging Face Transformers library:

- **T5Tokenizer**: Converts raw text into numerical token IDs that the T5 model can process. It also converts model outputs back into human-readable text.
- **T5ForConditionalGeneration**: The T5 neural network model itself, configured for 'conditional generation' tasks (text-in, text-out), which includes summarization, translation, and Q&A.

Cell 9 — Load FLAN-T5 Model

```
tokenizer = T5Tokenizer.from_pretrained("google/flan-t5-base")  
model = T5ForConditionalGeneration.from_pretrained("google/flan-t5-base")
```

Line 1: Downloads and loads the tokenizer for 'google/flan-t5-base' from the Hugging Face model hub. The tokenizer is specific to this model and uses SentencePiece tokenization.

Line 2: Downloads and loads the actual FLAN-T5 base model weights (~250MB). FLAN-T5 is a fine-tuned version of T5 trained on a large number of tasks using instruction-following data, making it excellent for zero-shot and few-shot prompting.

Cell 10 — Initialize Output Storage

```
output_list = []
```

Creates an empty Python list called `output_list`. This will store the generated summaries as the loop processes each conversation. Using a list and then assigning to a DataFrame column at the end is more efficient than modifying the DataFrame inside a loop.

Cell 11 — Generate Summaries Loop

```
for conv in df['Conversation']:  
    prompt = f"""  
    {conv}  
    What were the main points in that conversation?  
    """  
    input_ids = tokenizer(prompt, return_tensors="pt").input_ids  
    output_ids = model.generate(input_ids)  
    summary = tokenizer.decode(output_ids[0], skip_special_tokens=True)  
    output_list.append(summary)
```

This loop processes each conversation to generate a summary:

- Line 1: Iterates over each conversation text in the 'Conversation' column.
- Lines 2-5: Constructs a prompt using an f-string. The conversation text is inserted, followed by the instruction 'What were the main points in that conversation?' — this instruction-style prompting is what FLAN-T5 excels at.
- Line 6: Tokenizes the prompt. `return_tensors='pt'` returns PyTorch tensors. `.input_ids` extracts just the token ID tensor needed for the model.

- Line 7: Passes the tokenized input through the model to generate output tokens. `model.generate()` uses beam search by default.
- Line 8: Decodes the output token IDs back into human-readable text. `skip_special_tokens=True` removes tokens like `<pad>` and `</s>` from the output.
- Line 9: Appends the generated summary string to the `output_list`.

Cell 12 — Add Summaries to DataFrame

```
df['Summary'] = output_list  
df.head()
```

Line 1: Assigns the list of generated summaries as a new column 'Summary' in the DataFrame. Each summary corresponds to the conversation at the same row index.

Line 2: Displays the first 5 rows with the new Summary column to verify the results.

Cell 13 — Print All Summaries

```
for summary in df['Summary']:  
    print(summary)
```

Iterates over each summary in the 'Summary' column and prints it to the console. This provides a quick human-readable review of all 20 generated summaries to assess quality.

Task 3: Detecting Cancellation Requests

Cell 14 — Load Zero-Shot Classifier

```
from transformers import pipeline
classifier = pipeline("zero-shot-classification", model="MoritzLaurer/multilingual-MiniLMv2-L6-mnli-xnli")
```

Line 1: Imports the pipeline function from Transformers — a high-level API that bundles tokenizer and model together for common NLP tasks.

Line 2: Creates a zero-shot classification pipeline using the multilingual MiniLM model. Zero-shot classification means the model can classify text into categories it was never explicitly trained on — it uses Natural Language Inference (NLI) to determine if the text 'entails' a given label. The multilingual model supports many languages, making it robust for international customer service data.

Cell 15 — Define Classification Labels

```
labels = ['cancellation', 'other']
```

Defines the two classification categories as a Python list. The model will score each conversation against both labels and return the one with the highest confidence. 'cancellation' captures conversations where a customer is ending their subscription; 'other' captures all remaining interactions.

Cell 16 — Classify All Conversations

```
classification = []
for conv in df['Conversation']:
    result = classifier(conv, labels)
    classification.append(result['labels'][0])
```

Line 1: Creates an empty list to store classification results.

- Line 2: Iterates over each conversation text.
- Line 3: Runs the zero-shot classifier on the conversation with the two defined labels. The model returns a dictionary with 'labels' sorted by confidence score (highest first) and their corresponding 'scores'.
- Line 4: Appends only the top-scoring label (result['labels'][0]) to the classification list.

Cell 17 — Add Cancellation Boolean Column

```
df["Cancellation"] = [True if cls == 'cancellation' else False for cls in classification]
```

Uses a list comprehension to convert the string labels ('cancellation' or 'other') into Boolean values. True means the conversation was classified as a cancellation request; False means it was classified as 'other'. This Boolean column makes it easy to filter and analyze cancellation conversations in subsequent tasks.

Cell 18 — Display Updated DataFrame

```
df.head()
```

Displays the first 5 rows of the DataFrame, now including the new 'Cancellation' Boolean column alongside Date, Time, Conversation, and Summary columns.

Task 4: Identifying Cancellation Reasons

Cell 19 — Define cancellation_reasons Function

```
def cancellation_reasons(df):
    if df['Cancellation'] == False:
        return 'None'
    else:
        prompt = f"""
        {df['Conversation']}
        What are the issues that led the client to cancel their subscription?
        """
        input_ids = tokenizer(prompt, return_tensors="pt").input_ids
        output_ids = model.generate(input_ids, max_new_tokens=50, min_length=20)
        return tokenizer.decode(output_ids[0], skip_special_tokens=True)
```

This function conditionally extracts cancellation reasons using FLAN-T5:

- Lines 2-3: If the row's Cancellation flag is False, immediately returns the string 'None' — no AI processing needed for non-cancellation conversations.
- Lines 4-8: If the conversation IS a cancellation, constructs a targeted prompt asking specifically about the issues that led to cancellation. This instruction is more specific than the summarization prompt, directing FLAN-T5 to focus on negative experiences.
- Line 9: Tokenizes the prompt into PyTorch tensors.
- Line 10: Generates the output with max_new_tokens=50 (limits response length) and min_length=20 (ensures a minimum response length to avoid trivial outputs).
- Line 11: Decodes and returns the extracted reason as a string.

Cell 20 — Apply Function to Get Cancellation Reasons

```
df['Cancellation_reasons'] = df.apply(cancellation_reasons, axis=1)
df
```

Line 1: Applies the cancellation_reasons function to every row of the DataFrame using .apply() with axis=1 (row-wise). For non-cancellation rows, it returns 'None' immediately (fast). For cancellation rows (5 out of 20), it runs FLAN-T5 to extract the specific reason. The results are stored in a new 'Cancellation_reasons' column.

Line 2: Displays the full DataFrame (all 20 rows) showing all columns, including the final Cancellation_reasons column.

Task 5: Word Cloud Visualization

Cell 21 — Import Visualization Libraries

```
import matplotlib.pyplot as plt
from wordcloud import WordCloud
```

Line 1: Imports matplotlib's pyplot module under the alias 'plt'. Matplotlib is Python's primary plotting library; pyplot provides a MATLAB-like interface for creating figures and plots.

Line 2: Imports the WordCloud class from the wordcloud library. WordCloud generates visual representations of text where word size reflects frequency — more frequent words appear larger.

Cell 22 — Define generate_wordcloud Function

```
def generate_wordcloud(text_series, title):
    text = " ".join(review for review in text_series)
    wordcloud = WordCloud(width=800, height=400,
background_color='white').generate(text)
    plt.figure(figsize=(10,5))
    plt.imshow(wordcloud, interpolation='bilinear')
    plt.axis("off")
    plt.title(title)
    plt.show()
```

This reusable function generates and displays a word cloud from a pandas Series of text:

- Line 2: Concatenates all text entries in the series into a single string separated by spaces. This is necessary because WordCloud expects a single string, not a list.
- Line 3: Creates a WordCloud object with width=800px, height=400px, and white background, then calls .generate(text) to compute word frequencies and positions.
- Line 4: Creates a matplotlib figure with a 10x5 inch canvas.
- Line 5: Renders the word cloud image inside the matplotlib figure using imshow(). The interpolation='bilinear' parameter creates smooth edges on the word cloud image.
- Line 6: Removes the axis labels and tick marks — they are not meaningful for image plots.
- Line 7: Adds the specified title above the word cloud.
- Line 8: Displays the completed word cloud visualization.

Usage — Generate Word Clouds

```
generate_wordcloud(df['Conversation'], 'Word Cloud: Customer Conversations')
generate_wordcloud(df['Summary'], 'Word Cloud: Conversation Summaries')
```

These two calls generate word clouds for: (1) the full conversation texts — showing all language used across the 20 calls; (2) the AI-generated summaries — showing the most important terms after NLP condensation. Comparing both clouds reveals which topics the AI model prioritized in its summarization.

7. Conclusion

This project successfully demonstrated the power of combining Generative AI and pre-trained NLP models to extract meaningful insights from unstructured customer service data. The automated pipeline:

- Reduced manual analysis time from hours to minutes
- Accurately identified 5 cancellation conversations (100% recall confirmed by manual review)
- Extracted specific, actionable cancellation reasons without any additional model training
- Provided visual representations of key customer themes through word clouds

The pipeline is generalizable and can scale to handle hundreds or thousands of call logs, making it a valuable tool for ongoing customer service monitoring at Fresh Fare Meal Kits.

7.1 Future Enhancements

- Expand the cancellation label set to include sub-categories (e.g., 'delivery issue', 'quality issue', 'pricing')
- Integrate sentiment analysis to track customer emotion throughout the conversation
- Build a real-time dashboard for live call log monitoring
- Apply fine-tuned models on domain-specific meal kit data for higher accuracy

7.2 Technical Stack Summary

Component	Version/Model	Purpose
Python	3.12	Core language
pandas	latest	Data manipulation
Transformers	latest	NLP model loading
google/flan-t5-base	HF Hub	Summarization & reason extraction
MoritzLaurer/multilingual-MiniLMv2	HF Hub	Zero-shot cancellation detection
matplotlib	latest	Visualization
wordcloud	latest	Word cloud generation
torch	latest	Deep learning backend